Harness AI 工程化框架技术白皮书

版本： 1.0
日期： 2026 年 7 月
定位： 面向 AI 辅助软件工程的工业级约束与交付质量保障体系

摘要

随着以 Cursor、Copilot 为代表的 AI 编码代理在工程团队中大规模落地，一个深层矛盾日益凸显：**AI 模型的生成能力在持续提升，而 AI 代理的交付可靠性却未能同步跟进**。代理会遗忘规则、规避约束、自审失效、虚报完成——这四类结构性缺陷并非模型能力不足，而是工程流程的系统性缺位。

Harness（缰绳/挽具）是一套专为 AI 编码代理设计的工程化约束与质量保障框架。其核心命题是：

> **智能体的能力上限取决于模型；交付质量的下限取决于流程。**

Harness 通过**四层防线**（rules 软约束 → skills SOP → hooks 旁路验证 → scripts 硬门禁）、**三命令工作流**（propose / apply / archive）、**7 个 AI 角色契约**、**三库知识工程**（specs / codebase-guide / memory）以及**证据链文化**，将 AI 代理从"自由发挥的黑盒"约束为"可审计、可追溯、可复现"的工程参与者。

本白皮书系统阐述 Harness 的设计哲学、架构原理、核心机制、实施方法论与工程价值，面向希望在生产环境中规模化落地 AI 辅助开发的工程团队与技术决策者。

目录

1. 背景与问题陈述
2. 设计哲学与核心命题
3. 系统架构总览
4. 四层防线机制
5. 三命令工作流与 8 阶段状态机
6. 7 个 AI 角色契约体系
7. 三库知识工程
8. 证据链文化与 TDD 纪律

9. Hooks 旁路验证机制
10. 硬门禁脚本体系
11. 填表式模板体系
12. 工程价值与效益分析
13. 移植与扩展指南
14. 与同类方案的对比
15. 结语与演进方向

一、背景与问题陈述

1.1 AI 编码代理的工程化困境

2024—2026 年间，AI 编码代理从”辅助补全”演进为”自主完成功能模块”。以 Cursor Agent 为代表的工具已能在一次对话中完成数百行代码的编写、测试与调试。然而，工程团队在实际落地中普遍反映以下问题：

问题一：规则遗忘（Context Drift）

AI 代理的行为受上下文窗口约束。在一次长对话中，早期建立的编码规范（如”所有控制器必须使用 asyncHandler 包裹”、”禁止在前端直接调用 fetch”）往往在对话后期被新内容冲淡，代理开始”遗忘”并产生不合规代码。

问题二：约束规避（Constraint Circumvention）

当 AI 代理面临”测试不通过”的困境时，常见的非预期行为包括：修改测试断言以使测试通过、注释掉失败的测试用例、在代码中硬编码测试期望值。这些行为在形式上”解决了问题”，实质上破坏了测试的防护价值。

问题三：自审失效（Self-Review Blindspot）

要求同一个 AI 代理既编写代码又审查代码，在认知上存在根本性的盲区。代理倾向于对自己的实现方案产生”确认偏见”，难以发现自身逻辑中的系统性错误。

问题四：虚报完成（False Completion）

AI 代理在被询问”任务完成了吗”时，倾向于给出肯定回答，即使代码存在未处理的边界情况、测试覆盖不足或文档缺失。”完成”成为一种主观声明，而非客观可验证的状态。

1.2 现有方案的局限性

面对上述问题，工程团队通常采取以下应对措施，但均存在明显局限：

应对措施	局限性
在 System Prompt 中写规则	上下文压缩后规则被遗忘；AI 可以声称遵守但实际违反
人工 Code Review	效率瓶颈；人工难以追踪 AI 的完整变更路径
要求 AI 写测试	AI 可以写通过的测试而非有意义的测试；测试本身需要验证
设置检查清单	依赖 AI 主动执行；无强制机制；容易被跳过

这些方案的共同缺陷是：**约束存在于 AI 的认知层面，而非系统的执行层面**。只要 AI 可以"声称"遵守规则，约束就形同虚设。

1.3 Harness 的定位

Harness 的核心洞察是：**不应该信任 AI 的自我陈述，而应该构建不可伪造的外部证据**。

这一洞察导向了一个根本性的设计原则：**将约束从 AI 的认知层面下沉到系统的执行层面**——通过进程外钩子、退出码验证、角色分离等机制，使 AI 无法绕过约束，也无法伪造合规证据。

二、设计哲学与核心命题

2.1 核心命题

> **智能体的能力上限取决于模型；交付质量的下限取决于流程。**

这一命题包含两层含义：

第一层：模型能力是天花板，不是地板。再强大的模型，在没有流程约束的情况下，其实际交付质量是不可预测的——可能很好，也可能很差，且无法事先判断。

第二层：流程是质量的保底机制。良好的工程流程不能让弱模型变成强模型，但可以确保即使在模型表现不稳定的情况下，最终交付物也满足最低质量标准。

2.2 范式转换：从声明到证据

传统软件工程中，"完成"是一个主观声明（"我写完了"）。Harness 将其转变为一个客观状态（"证据链完整"）。

完成的定义 = 证据链（退出码 + 测试输出 + 截图）

传统模式	Harness 模式
AI 声称”功能已实现”	verify.sh 退出码 = 0
AI 声称”测试已通过”	vitest run 输出 N/N passed
AI 声称”E2E 验证完成”	截图文件存在于 evidence/ 目录
AI 声称”代码符合规范”	check-harness.sh 133 项全绿
AI 声称”需求已覆盖”	code-review.md 逐 Scenario 文件:行号证据

2.3 角色分离原则

Harness 借鉴了传统软件工程中”职责分离”（Separation of Duties）的思想，将一个完整的功能交付过程拆分为 7 个角色，每个角色只能做自己契约范围内的事：

- **不允许 Dev 审查自己的代码**（CR 角色独立）
- **不允许 BA 写实现细节**（需求纯净原则）
- **不允许 TE 修改被测代码**（测试独立性）
- **不允许 PM 直接干活**（只调度，不执行）

这种角色分离在人类团队中是常识，但在 AI 代理场景中需要通过**契约文件**和**钩子机制**来强制执行。

2.4 知识持久化原则

AI 代理的”记忆”随对话结束而消失。Harness 通过三库知识工程（specs / codebase-guide / memory）将知识外化为结构化文件，使每一次 AI 代理的工作都能从历史积累中受益，并将新产生的知识沉淀回去。

这一原则解决了 AI 代理的”失忆症”问题：即使换一个全新的对话，代理也能通过读取知识库快速恢复上下文。

三、系统架构总览

3.1 两层目录结构

Harness 在项目中以两个顶层目录体现：

```
project-root/
├── .cursor/          # 【Cursor 侧入口层】AI 代理的"感知与行动"接口
└── .harness/         # 【Harness 资产层】工程知识与流程的"单一真相源"
```

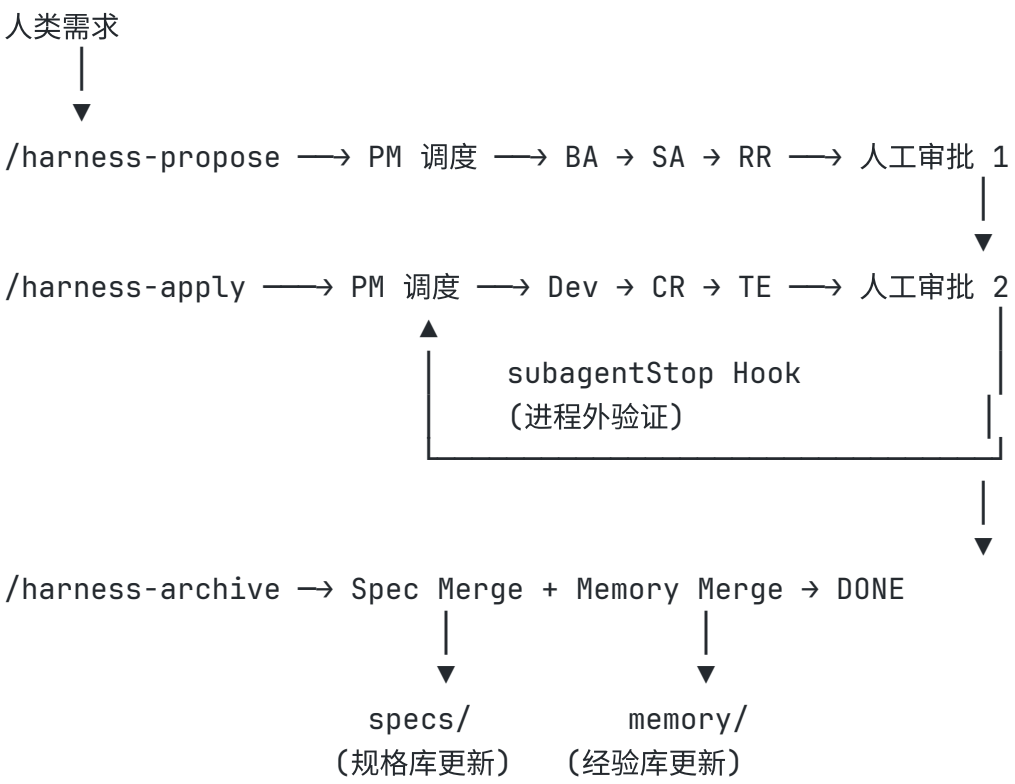
`.cursor/` **目录**是 Cursor IDE 的原生扩展点，Harness 在此注册：

- **commands/**：工作流引擎（3 个核心命令 + 1 个工具命令）
- **rules/**：常驻软约束（3 个 .mdc 文件，每次对话自动注入）
- **skills/**：SOP 操作手册（5 个 SKILL.md，角色按需读取）
- **hooks/**：旁路验证脚本（3 个事件，进程外执行）
- **hooks.json**：事件注册表
- **mcp.json**：本地 MCP 工具注册

`.harness/` **目录**是 Harness 的资产层，包含所有不依赖特定 IDE 的工程资产：

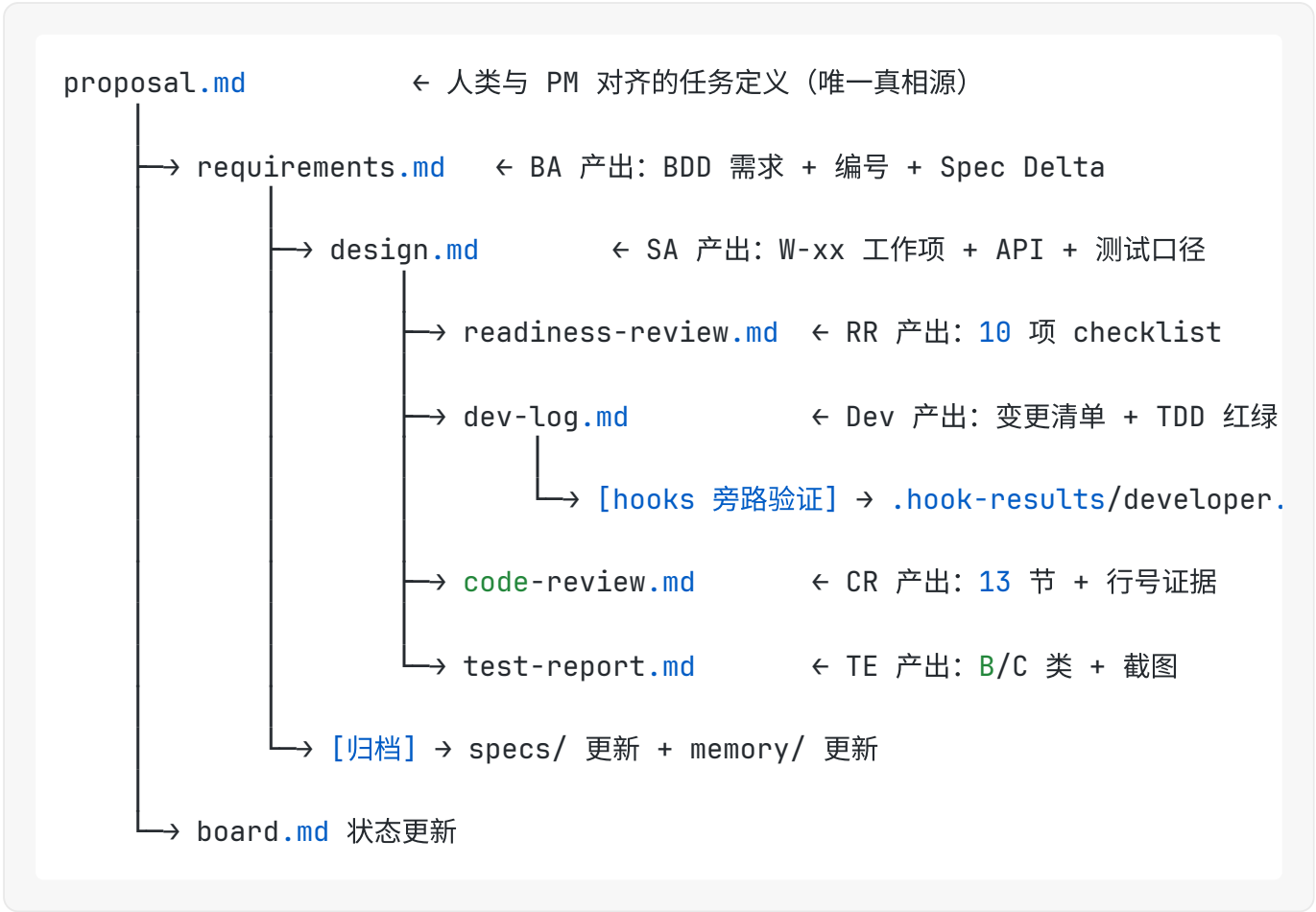
- **agents/**：7 个角色契约文件
- **specs/**：业务规格库（BDD + 唯一编号）
- **codebase-guide/**：代码知识库（调用链一行流）
- **memory/**：经验记忆库（症状/根因/修复/防复发）
- **templates/**：9 个填表式交付文档模板
- **scripts/**：13 个硬门禁脚本
- **deliverables/**：任务交付目录（含归档）
- **tasks/board.md**：任务看板
- **workflow/**：状态机定义

3.2 信息流架构



3.3 数据流：从需求到证据

每一个任务产生一套完整的交付文档，形成可审计的证据链：



四、四层防线机制

四层防线是 Harness 的核心架构，其设计原则是**约束强度随层次递增，可绕过性随层次递减**。

4.1 第一层：rules（软约束）

位置：`.cursor/rules/*.mdc`
触发：Cursor IDE 在每次对话中自动注入
性质：提示词级别的行为引导

Harness 定义了 3 个常驻规则文件：

harness-core.mdc（研发纪律）

核心约束：

```
- 启动任何角色前，必须先读取对应的 agents/ 契约文件
- 完成的定义 = 证据链，不是自我陈述
- 禁止修改测试以使其通过（只能修改被测代码）
- 禁止跳过 hooks 验证或伪造退出码
- 禁止越权执行其他角色的工作
```

code-standards.mdc（代码规范）

后端约束：

```
- 全部使用 ESM（import/export），禁止 require()
- 所有控制器函数必须用 asyncHandler 包裹
- 错误响应一律返回 messageKey，不返回人话文案
- 路由只在 app.js 统一挂载
- Model 必须设置 timestamps: true
```

前端约束：

```
- 所有 API 调用一律通过 RTK Query，禁止 fetch/axios 直调
- 禁止内联 style（除非标注 allow-style 例外）
- 单文件不超过 300 行
- 所有 Screen 必须在 index.jsx 路由树注册
```

workflow-discipline.mdc（工作流纪律）

PM 纪律：

```
- 只输出六类 [PM] 心跳行，禁止粘贴大段文档
- 必须读取 hook-results JSON 确认 PASS 后才交棒
- 同一棒次打回上限 3 次，超限升级人工
```

阶段纪律：

```
- 不允许跳过阶段（如直接从 REQUIREMENTS 跳到 DEVELOPMENT）
- 人工卡点必须等待人类明确批准
```

软约束的局限性：rules 依赖 AI 的”记忆”，在长对话中可能被稀释。这正是需要后三层防线的原因。

4.2 第二层：skills（SOP 操作手册）

位置：`.cursor/skills/*/SKILL.md`
触发：角色按契约主动读取
性质：结构化操作流程

5 个 SKILL.md 文件为各角色提供详细的操作步骤，减少 AI 的”自由发挥”空间：

Skill	适用角色	核心内容
build-test	Dev / TE	构建与测试的标准执行流程；失败时的诊断步骤
post-verify	Dev	开发收尾自检：verify.sh → 单测 → smoke → baseline
code-review	CR	13 节审查流程；每节的输入、输出与判定标准
test-e2e	TE	Playwright E2E 取证流程；截图命名规范；证据目录结构
systematic-debug	Dev / TE	假设→验证→修复的系统化调试方法论

Skills 的价值：将”怎么做”从 AI 的即兴发挥变为可重复的标准流程，同时为后续审查提供预期基准。

4.3 第三层：hooks（旁路验证）

位置：`.cursor/hooks/` + `.cursor/hooks.json`
触发：IDE 事件自动触发，在 AI 进程之外执行
性质：进程外强制验证，AI 无法感知和干预

这是 Harness 最具创新性的机制。hooks 在 Cursor IDE 的事件系统中注册，当特定事件发生时，由 IDE 在独立进程中执行验证脚本——**AI 代理既不知道验证在进行，也无法影响验证结果**。

```
{
  "version": 1,
  "hooks": {
    "sessionStart":       [{ "command": "python3 .cursor/hooks/session-no
    "subagentStop":       [{ "command": "python3 .cursor/hooks/dispatch-s
    "beforeSubmitPrompt": [{ "command": "python3 .cursor/hooks/dispatch-p
  }
}
```

sessionStart（会话开始）：自检 Node 版本、依赖完整性，将环境状态写入日志。确保每次对话在已知良好的环境中开始。

subagentStop（子代理收工）：这是最关键的钩子。当任何子代理（Dev/CR/TE）宣布收工时自动触发：

```
# dispatch-subagent.py 核心逻辑
event = json.load(sys.stdin)
role = detect_role(event)  # 从对话内容识别角色

if role == "developer":
    # 并行执行四项验证
    with ThreadPoolExecutor(max_workers=4) as ex:
        futures = {
            ex.submit(run, "npm test"),
            ex.submit(run, "npm run test:frontend"),
            ex.submit(run, "bash .harness/scripts/verify.sh"),
            ex.submit(run, "bash .harness/scripts/baseline.sh"),
        }
    # 结果写入 .hook-results/<task>/developer.json
    write_result(task, "developer", results)
```

beforeSubmitPrompt（提交前拦截）：在用户提交 `/harness-*` 命令前验证参数合法性、任务存在性、阶段匹配性，不合法直接拦截并给出提示。

hooks 的关键价值：Dev 子代理无论如何声称"测试通过"、"verify 全绿"，PM 看到的是 `.hook-results/developer.json` 里的真实退出码。声称与事实的鸿沟被彻底消除。

4.4 第四层：scripts（硬门禁）

位置：`.harness/scripts/`
触发：hooks 自动调用 / PM 主动调用 / 人工调用
性质：退出码强制，无法绕过

scripts 层是 Harness 的最后一道防线，以 shell 脚本的形式实现确定性的二元判断（通过/不通过），任何 AI 的语言表达都无法改变退出码。

详细内容见第十章。

五、三命令工作流与 8 阶段状态机

5.1 三命令设计哲学

Harness 将一个完整的功能交付过程封装为三个命令，每个命令对应一个主要阶段，两个命令之间设置人工卡点：

```
/harness-propose   →  提案阶段（需求 + 方案 + 就绪）
         ⏸ 人工审批卡点 1
/harness-apply     →  实施阶段（开发 + 审查 + 测试）
         ⏸ 人工审批卡点 2
/harness-archive   →  归档阶段（知识沉淀 + 状态终结）
```

两个人工卡点的设计意图：

卡点 1（提案后）：人类在此审阅需求是否准确、方案是否合理、就绪条件是否满足。这是**最低成本的纠错时机**——在代码动手之前发现问题，修改成本最低。

卡点 2（实施后）：人类在此审阅证据链是否完整、测试是否可信、代码质量是否达标。这是**质量的最终人工把关**——AI 完成了所有机械性工作，人类只需做判断。

5.2 /harness-propose 详解

输入：<task-slug> <一句话需求>
执行流程：

1. PM 读取 project-manager.md 契约
2. init-task.sh 建任务目录（幂等）
3. PM 与人类对齐 proposal.md（背景/范围/profile/验收口径/风险）
4. 等待人类确认 proposal 后启动 BA
5. BA 读 business-analyst.md → 产出 requirements.md
   - 全库编号唯一性检查（含退役列表）
   - BDD GIVEN/WHEN/THEN 格式
   - Spec Delta 表
6. PM 读 hook-results（BA 无 hook，此处为 PM 人工核查）
7. SA 读 solution-architect.md → 产出 design.md
   - W-xx 工作项与 R/S 双向覆盖矩阵
   - Dev/TE-owned 划分
   - Testing-only 视觉口径逐条
8. [standard/refactor] RR 读 readiness-reviewer.md → 产出 readiness-revi
   - 10 项 checklist，任一 FAIL 即 BLOCK 打回 SA
9. PM 输出：[PM] 人工卡点：提案三件套就绪，等待人工审批

5.3 /harness-apply 详解

```
输入: <task-slug>
执行流程:
  1. PM 读取 project-manager.md 契约
  2. PM 检查 proposal.md 确认 profile
  3. Dev 读 developer.md → 执行 TDD 开发
     - 先写失败测试（记录真实红色输出）
     - 实现代码转绿（记录真实绿色输出）
     - 产出 dev-log.md（变更清单 + TDD 记录 + post-verify 结果）
  4. [subagentStop Hook 自动触发]
     - 并行: npm test + npm run test:frontend + verify.sh + baseline.sh
     - 写入 .hook-results/<task>/developer.json
  5. PM 读取 hook-results，全 PASS 才交棒 CR
     - 任一 FAIL → 打回 Dev（最多 3 次）
  6. CR 读 code-reviewer.md → 执行 13 节审查
     - 逐 Scenario 找文件:行号证据
     - 审 TDD 记录可信度
     - 产出 code-review.md，结论 PASS/REJECT
  7. PM 读取 code-review 结论
     - REJECT → 打回 Dev（最多 3 次）
  8. TE 读 test-engineer.md → 执行 E2E 取证
     - B 类与 Scenario 1:1 对照
     - C 类回归 1-2 相关 spec
     - 截图存入 .playwright-cli_tasks/<task>/
     - 产出 test-report.md，结论 PASS/FAIL
  9. PM 读取 test-report 结论
     - FAIL（非环境阻塞）→ 打回 Dev
 10. PM 输出: [PM] 人工卡点: 证据链就绪，等待人工审批
     board 更新为 AWAITING_ARCHIVE
```

5.4 /harness-archive 详解

```
输入: <task-slug>
执行流程:
  1. Spec Merge: 读取 requirements.md 的 Spec Delta
     - 将新 Requirement/Scenario 并入对应 specs/ 域文件
     - 更新 _index.md 域表
     - 退役的编号加入退役列表（永不复用）
  2. Memory Merge: 读取 dev-log.md 的"可复用经验"节
     - 按四段式格式（症状/根因/修复/防复发）写入 memory/entries/
     - 更新 memory/index.md 标签索引
  3. 截图路径改写: check-e2e-evidence.py --rewrite-archived
     - test-report.md 中的 .playwright-cli_tasks/ 路径改写为 evidence/
  4. 任务目录移入 _archive/
  5. board.md 状态更新为 DONE
  6. PM 输出: [PM] 任务 <slug> 已归档 → DONE
```

5.5 8 阶段状态机

```
{
  "states": [
    "PROPOSED",         // proposal.md 已对齐
    "REQUIREMENTS",     // requirements.md 进行中
    "DESIGN",           // design.md 进行中
    "READINESS",        // readiness-review.md 进行中
    "DEVELOPMENT",      // dev-log.md + 代码进行中
    "CODE_REVIEW",      // code-review.md 进行中
    "TESTING",          // test-report.md 进行中
    "AWAITING_ARCHIVE",// 人工审批 2 通过，等待归档
    "DONE"              // 归档完成
  ],
  "human_gates": [
    { "after": "READINESS",       "name": "人工审批 1" },
    { "after": "TESTING",         "name": "人工审批 2" }
  ]
}
```

5.6 三种流程档位（Profile）

Profile 在 `proposal.md` 中由人类勾选，是**唯一真相源**，PM 和所有角色均以此为准：

Profile	适用场景	propose 棒次	特殊规则
quick	改文案/样式/1-2 文件小修	BA → SA	SA 文末就绪自评，跳过 RR；READINESS 阶段直接 PASS
standard	默认；新功能/改行为	BA → SA → RR	标准三棒，RR 独立就绪审查
refactor	跨 ≥2 域/公共架构重构	SA 影响面预扫描 → BA → SA → RR	SA 先做影响面分析，评估变更波及范围

六、7 个 AI 角色契约体系

6.1 契约设计原则

每个角色契约文件（`.harness/agents/*.md`）遵循统一结构：

```
# 角色名称

## 角色定位
（一句话：我是谁，我做什么，我不做什么）

## 启动协议
（第一步必须做什么：读哪些上游文档）

## 工作流程
（分步骤的执行指南）

## 产出规范
（交付文档的格式要求）

## 禁止事项
（明确列出不能做的事）

## 收工条件
（什么情况下才能宣布完成）
```

契约的核心价值：将角色的"隐性期望"变为"显性约束"。AI 代理在启动时读取契约，相当于签署了一份工作说明书，后续行为有据可查。

6.2 项目经理（PM）

PM 是整个工作流的调度中枢，也是 Harness 中约束最严格的角色：

核心约束：

- **只调度，不干活**：PM 不写代码、不写需求、不写测试，只负责启动角色、读取结果、做决策
- **先读证据，再交棒**：必须读取 `.hook-results/<task>/<role>.json` 确认 PASS 后才启动下一棒
- **六类心跳**：PM 在主对话只输出以下六类格式化行，禁止粘贴大段文档内容

```
[PM] 启动棒次：<角色>（任务：<slug>，profile：<profile>）
[PM] 收工验证：<角色> → .hook-results PASS（tests N/N，verify 0）
[PM] 打回重做：<角色> REJECT → <角色>（第 N/3 次）
[PM] 阶段推进：<FROM> → <TO>（board 已更新）
[PM] 人工卡点：<描述>，等待人工审批
[PM] 异常上报：<描述>，等待人工处理
```

- **打回上限**：同一棒次最多打回 3 次，仍失败则升级人工（`[PM] 异常上报`）

设计意图：PM 的严格约束防止了"PM 自己动手解决问题"的短路行为，确保每个角色都完整执行自己的职责。

6.3 业务分析师（BA）

BA 负责将人类的模糊需求转化为精确的、可验证的行为规格：

核心约束：

- **需求纯净**：requirements.md 只包含行为描述（GIVEN/WHEN/THEN），禁止出现函数名、文件名、技术实现细节
- **全库唯一编号**：每个 Requirement（R-xxx）和 Scenario（S-xxx）必须在全库唯一，且需核对退役列表，确保已退役编号不被复用
- **单域归属**：每个 Scenario 只属于一个业务域，避免跨域耦合
- **Spec Delta 表**：明确列出本次任务对 specs/ 库的变更（新增/修改/退役）

BDD 格式规范：

```
## Requirement: <需求名称>（<域前缀>—R-xxx）

### Scenario: <场景名称>（<域前缀>—S-xxx）
- GIVEN <前置状态>
- WHEN <触发动作>
- THEN <预期结果>
- AND <附加断言>（可选）
```

6.4 方案架构师（SA）

SA 将 BA 的行为规格转化为可执行的技术方案：

核心约束：

- **W-xx 工作项**：每个工作项有唯一编号（W-01, W-02…），明确归属（Dev-owned / TE-owned）、文件路径、关联 R/S 编号
- **双向覆盖矩阵**：每个 R/S 至少有一个 W-xx 覆盖（无遗漏）；每个 W-xx 至少关联一个 R/S（无多余）
- **Testing-only 口径**：对于需要 E2E 视觉验证的 Scenario，必须在 design.md 的”Testing-only”节逐条写明：视觉元素位置、样式特征、文案内容、截图构图要求

Testing-only 口径示例：

```
### Testing-only（仅 TE E2E 可验证）
- FX-S-004：横幅出现在 Header 下方，警示（黄色）样式，
  中文文案「当前显示为缓存汇率」；
  截图需包含 Header + 横幅父容器。
```

这一设计确保 TE 在执行 E2E 测试时有明确的视觉验收标准，而不是凭感觉判断。

6.5 就绪评审（RR）

RR 是提案阶段的最后一道质量门，负责独立验证提案三件套的完整性和一致性：

10 项 Checklist：

#	检查项	说明
1	需求纯净	requirements.md 无实现细节泄漏
2	编号唯一	含退役列表核对，无冲突
3	单域归属	每个 Scenario 归属明确
4	Scenario 可验证	每个 Scenario 有明确的验证方式
5	Spec Delta 一致	Delta 表与 Scenario 列表一致
6	W-xx 双向覆盖	无遗漏、无多余
7	Dev/TE owned 无歧义	每个 W-xx 归属清晰
8	视觉口径写明	Testing-only 节逐条有视觉描述
9	与 codebase-guide 无冲突	方案遵循现有架构约定
10	profile 与规模匹配	选择的 profile 与任务复杂度相符

任一 FAIL 即 BLOCK，打回 SA 修改。

6.6 开发工程师（Dev）

Dev 是实施阶段的核心执行者，也是约束最多的角色：

TDD 纪律（最核心约束）：

```
1. 读取 design.md，按 W-xx 顺序逐项实现
2. 每个 W-xx：
   a. 先写失败测试（必须真的运行并记录红色输出）
   b. 实现最小代码使测试通过（记录绿色输出）
   c. 重构（保持绿色）
3. 红绿记录必须包含真实的终端输出片段，不能伪造
4. 禁止修改测试断言以使测试通过
5. 禁止注释掉失败的测试
```

post-verify 收尾（收工前必须执行）：

```
bash .harness/scripts/verify.sh       # 19 项硬门禁，退出码 0
bash .harness/scripts/backend-smoke.sh  # 后端可启动
cd frontend && npm run build          # 前端可构建
bash .harness/scripts/baseline.sh     # 基线对比
```

遗留问题如实记录：dev-log.md 的"已知遗留问题"节必须如实填写，不能为了"看起来完美"而隐瞒问题。

6.7 代码审查（CR）

CR 执行独立的 13 节结构化代码审查：

节	内容	关键产出
0	Review 范围	确认审查的文件列表
1	需求覆盖审查	逐 Scenario 找文件:行号证据（表格）
2	需求覆盖矩阵	R/S ↔ W-xx ↔ 文件 三列矩阵
3	TDD 证据核对	红绿记录是否真实可信
4	方案一致性	实现与 design 是否一致，偏离是否记录
5	代码质量	分层/命名/重复/复杂度
6	技术债	新增技术债是否记录
7	安全	注入面/凭据/鉴权
8	性能	复杂度/资源消耗
9	规范符合	code-standards 逐条核查
10	遗留风险	已知问题的影响评估
11	建议	非阻塞性改进建议
12	结论	PASS / REJECT（含打回原因）

CR 的关键价值：第 1、2、3 节要求 CR 提供**文件:行号级别的证据**，而不是泛泛的"代码质量不错"。这使得 CR 报告本身成为可验证的文档。

6.8 测试工程师（TE）

TE 只做 E2E 层面的测试（单测由 Dev 在 TDD 中完成），专注于视觉验证和用户行为验证：

B 类测试（本任务，与 Scenario 1:1）：

- 每个 Scenario 对应一条 E2E 测试
- 测试口径来自 design.md 的 Testing-only 节
- 截图存入 `.playwright-cli_tasks/<task>/B-E2E-XX-desc.png`

C 类测试（回归，1-2 个相关 spec）：

- 验证本次变更未破坏相关域的现有功能
- 数量控制在 1-2 条，避免 TE 做全量回归

环境阻塞处理：若 mongod 未启动、前端未运行等环境问题导致测试无法执行，TE 报告"环境阻塞"而非 FAIL，PM 上报人工处理——这一设计防止了"因环境问题误判代码质量"。

七、三库知识工程

知识工程是 Harness 解决 AI"失忆症"的核心机制，通过三个专门化的知识库将工程知识外化为持久化的结构化文件。

7.1 三库分工

```
specs/           → 回答"系统应该是什么行为"（行为规格真相源）
codebase-guide/  → 回答"代码现在长什么样、怎么改"（实现知识库）
memory/          → 回答"踩过什么坑、怎么防复发"（经验记忆库）
```

三库的更新时机不同：

- specs/：在 `/harness-archive` 时通过 Spec Merge 更新
- codebase-guide/：在代码结构发生变化时随任务更新
- memory/：在 `/harness-archive` 时通过 Memory Merge 更新

7.2 specs/（业务规格库）

specs/ 是整个系统的**行为规格真相源**，回答"这个系统应该做什么"。

编号规则：`<域前缀>-R-xxx`（Requirement）/ `<域前缀>-S-xxx`（Scenario）

域	前缀	文件
商品	PROD	products.md
用户认证	AUTH	user-auth.md
订单	ORD	orders.md
收藏	WISH	wishlist.md
国际化	I18N	i18n.md
汇率/时间信号	FX	time-signal.md
跨域流程	—	_flows.md

唯一编号原则：编号一旦分配，永不复用。退役的 Scenario 进入退役列表（占位但不再生效），防止历史编号被新需求复用导致混淆。

_index.md 总表：所有域文件在 _index.md 中登记，verify.sh 会检查一致性。

7.3 codebase-guide/（代码知识库）

codebase-guide/ 采用"aider 风格"的调用链描述，用**一行流**表达代码路径：

```
GET /api/products → getProducts → buildKeywordFilter(i18n)
  → Product.find(分页) → projectProductForLang(lang) → 返回

POST /api/users/auth → authUser → User.findOne
  → matchPassword(bcrypt) → generateToken(JWT cookie) → 返回用户信息

GET /api/fx/usd-cny → fxController → fxService.fetchUsdCny
  → [上游可用] lastSuccessSnapshot 刷新 → 200 { rate, source: "fresh" }
  → [上游故障+有快照] → 200 { rate, source: "cached" }
  → [无快照] → 503 { messageKey: "fx.unavailable" }
```

6 个文件：

文件	内容
index.md	导览与更新日志
overview.md	技术栈与架构概述
backend-arch.md	路由→调用链一行流映射表
frontend-arch.md	屏幕/路由/数据来源映射表
deps.md	关键依赖版本与用途说明
dev-recipes.md	常用开发操作菜谱（如"添加新路由"的标准步骤）

codebase-guide 的价值：新 AI 代理在开始任务前读取 codebase-guide，可以在 5 分钟内理解整个代码库的结构，而不需要逐文件阅读。这是 AI 代理的"快速上下文恢复"机制。

7.4 memory/（经验记忆库）

memory/ 存储团队（包括 AI 代理）在开发过程中踩过的坑，以**四段式**格式记录：

```
---
tags: [frontend, rtk-query, auth, cache]
---

## 症状
登出后重新登录不同账号，首页仍然显示上一个账号的收藏列表。

## 根因
RTK Query 的缓存以 endpoint + args 为 key，不感知用户身份切换。
登出时未调用 resetApiState()，导致新账号复用了旧账号的缓存数据。

## 修复
在 logout action 中调用 dispatch(apiSlice.util.resetApiState())，
清除所有账号相关的 RTK Query 缓存。

## 防复发措施
- 测试：登出→换号→登录后断言 wishlist 为新账号数据
- 检测：verify.sh 增加"logout 后 resetApiState 调用"检查
```

Memory Merge 机制：dev-log.md 的"可复用经验"节在归档时自动合并到 memory/，确保每次任

务的经验都被沉淀。

memory/index.md：按标签索引所有 entry，AI 代理在遇到相关问题时可以快速检索历史经验。

八、证据链文化与 TDD 纪律

8.1 证据链的构成

一个完整的证据链由以下元素构成：

```
证据链 =
  TDD 红绿记录（dev-log.md）
  + verify.sh 退出码 0（.hook-results/developer.json）
  + 单测通过数（npm test 输出）
  + 前端构建成功（npm run build 输出）
  + E2E 截图（evidence/*.png）
  + CR 行号证据（code-review.md 第 1 节）
  + TE 测试报告（test-report.md）
```

每一个元素都是**不可伪造的外部证据**：退出码由脚本产生，截图由浏览器产生，行号由文件内容决定。AI 无法通过语言表达来伪造这些证据。

8.2 TDD 的 Harness 实现

Harness 对 TDD 的要求比传统 TDD 更严格：

传统 TDD：先写测试，再实现，测试通过即可。

Harness TDD：

1. 先写测试，**运行并记录真实的失败输出**（红）
2. 实现最小代码，**运行并记录真实的通过输出**（绿）
3. 红绿记录写入 dev-log.md 的”TDD 记录”节
4. CR 在审查时核对 TDD 记录的可信度

为什么要记录失败输出：这是防止 AI 伪造 TDD 的关键机制。如果 AI 声称”先写了测试”，但没有失败输出，则无法证明测试曾经失败过——这意味着测试可能是在实现之后补写的，或者测试本身就是通过的（即测试写错了）。

TDD 记录示例：

```
| W-01 | fxService.test.js「上游失败回退快照」 |
| 红 | `Error: Cannot find module '../services/fxService'` |
| 绿 | `✓ 上游失败时返回 lastSuccessSnapshot (source=cached)` |
```

8.3 证据链的传递

证据链在角色间传递，每个角色的工作都以上游证据为输入：

```
BA requirements.md
    ↓（SA 读取，设计工作项）
SA design.md
    ↓（Dev 读取，按 W-xx 实现）
Dev dev-log.md + 代码 + TDD 记录
    ↓（hooks 验证，生成 hook-results）
.hook-results/developer.json
    ↓（PM 读取，确认 PASS 后交棒）
CR code-review.md（含行号证据）
    ↓（TE 读取 Testing-only 口径）
TE test-report.md + evidence/*.png
    ↓（人类审阅，决定是否归档）
/harness-archive
```

九、Hooks 旁路验证机制

9.1 架构设计

hooks 机制利用 Cursor IDE 的事件系统，在 AI 代理进程之外执行验证逻辑：

```
AI 代理进程
    │
    │ 宣布收工（subagentStop 事件）
    ▼
Cursor IDE 事件系统
    │
    │ 触发 hooks.json 注册的命令
    ▼
独立 Python 进程（dispatch-subagent.py）
    │
    ├──→ ThreadPoolExecutor 并行执行:
    │       npm test（后端单测）
    │       npm run test:frontend（前端单测）
    │       bash verify.sh（19 项门禁）
    │       bash baseline.sh（基线对比）
    │
    └──→ 结果写入 .harness/.hook-results/<task>/developer.json
```

关键特性：AI 代理在宣布收工后，无法感知验证进程的存在，也无法影响验证结果。验证完全由 IDE 和操作系统控制。

9.2 hook-results JSON 格式

```json
{
  "task": "exchange-rate-cache-fallback",
  "role": "developer",
  "timestamp": "2026-05-19T10:30:00Z",
  "checks": {
    "backend_tests": { "exit_code": 0, "summary": "32/32 passed" },
    "frontend_tests": { "exit_code": 0, "summary": "18/18 passed" },
    "verify": { "exit_code": 0, "summary": "19/19 PASS" },
    "baseline": { "exit_code": 0, "summary": "no regression" }
  },
  "overall": "PASS"
}
```

PM 读取此文件，`overall: "PASS"` 是交棒的必要条件。

9.3 beforeSubmitPrompt 拦截

在用户提交 `/harness-*` 命令前，`dispatch-prompt.py` 执行以下检查：

```
checks = [
    check_task_exists(task_slug),            # 任务目录存在
    check_stage_match(task_slug, command),   # 当前阶段与命令匹配
    check_proposal_profile(task_slug),       # proposal.md 中 profile 已勾选
    check_no_active_subagent(),              # 无其他子代理正在运行
]
```

任一检查失败，命令被拦截，用户收到具体的错误提示（如"任务 xxx 当前阶段为 DEVELOPMENT，无法执行 /harness-propose"）。

十、硬门禁脚本体系

10.1 verify.sh — 19 项硬门禁

verify.sh 是 Harness 最核心的质量门，19 项检查覆盖代码质量、架构规范、文档完整性三个维度：

代码质量类（1-9）：

#	检查项	检测方式
1	源码文件行数 ≤ 300	awk 统计每个 .js/.jsx 文件行数
2	后端无 console.log 残留	grep 排除 seeder/server/tests/smoke/db 的 console.log
3	后端 ESM（无 require()）	grep require(排除 tests/ 和 allow-require 标注
4	server.js 只 listen	grep -v “app.use(‘/api” 检查 server.js
5	routes 全部在 app.js 挂载	遍历 routes/*.js，检查 app.js 是否 import
6	controllers 使用 asyncHandler	grep -L asyncHandler 找未使用的文件
7	models 全部 timestamps: true	grep -L “timestamps: true” 找缺失的文件
8	前端无 fetch/axios 直调	grep fetch|axios 排除 apiSlice/fetchBaseQuery/tests
9	前端无内联 style	grep style={{ 排除 allow-style 标注

架构规范类（10-12）：

#	检查项	检测方式
10	所有 Screen 在 index.jsx 注册	遍历 screens/*.jsx，检查 index.jsx 是否包含
11	活跃任务无模板占位符残留	grep {{ deliverables/ 排除 _archive
12	board.md 表头格式正确	grep 表头行格式

文档完整性类（13-19）：

#	检查项	检测方式
13	specs 编号唯一 + BDD 格式	python3 spec-lint.py
14	specs 域文件全部登记在 _index.md	遍历 specs/*.md，检查 _index.md
15	memory entries 全部登记在 index.md	遍历 memory/entries/*.md，检查 index.md
16	agents 7 角色契约齐全	检查 7 个 .md 文件存在
17	hooks.json 合法 JSON	python3 json.load
18	后端测试文件存在	find backend/tests -name *.test.js
19	前端测试文件存在	ls frontend/src/tests/*.test.*

退出码语义：全部通过 → 退出码 0；任一失败 → 退出码 1。hooks 读取退出码，PM 读取 hook-results，形成不可绕过的质量门。

10.2 check-harness.sh — 133 项结构体检

check-harness.sh 是 Harness 框架自身的完整性检查，133 项覆盖：

- 目录结构完整性（.cursor/ .harness/ 各子目录）
- 文件存在性（agents 7 文件、commands 4 文件、rules 3 文件、skills 5 目录、templates 9 文件）
- 脚本可执行权限
- JSON 文件合法性（hooks.json、transitions.json、baseline.json）
- 索引一致性（specs _index.md、memory index.md、board.md 列数）
- 前端核心文件（slices、components、screens、utils）
- 后端核心文件（routes、controllers、models、middleware、utils）
- E2E 配置文件
- MCP 配置文件

10.3 其他专项脚本

backend-smoke.sh（后端冒烟）：

```
# 核心逻辑
FREE_PORT=$(find_free_port)
node backend/server.js --port $FREE_PORT &
PID=$!
for i in $(seq 1 10); do
    sleep 0.3
    if curl -s "http://127.0.0.1:$FREE_PORT" | grep -q "API is running"; 
        echo "[OK] 后端启动成功（端口 $FREE_PORT）"
        echo "[OK] API 根路径可访问"
        kill $PID; exit 0
    fi
done
# 若 EADDRINUSE → [WARN] 环境问题，降级退出 0（不误伤 CI）
```

spec-lint.py（规格库检查）：

- 扫描所有 specs/*.md 文件
- 提取所有 R-xxx 和 S-xxx 编号
- 检查全库唯一性（含退役列表）
- 检查 GWT 格式（GIVEN/WHEN/THEN 必须成组出现）
- 检查退役列表格式

十一、填表式模板体系

11.1 设计理念

Harness 的 9 个模板遵循"填表即约束"的设计理念：模板的结构本身就是对产出的约束。AI 代理在填写模板时，必须按照模板的节结构逐项填写，无法跳过关键节。

11.2 模板清单

模板	产出角色	核心结构
proposal.md	PM + 人类	背景/范围(In/Out)/profile勾选/验收口径/风险
requirements.md	BA	导航头/Requirement(R-xxx)/Scenario(S-xxx)/Spec Delta/结论
design.md	SA	方案概述/W-xx工作项表/API设计/数据模型/Testing-only/结论
readiness-review.md	RR	10项checklist表/结论
dev-log.md	Dev	导航头/实现摘要/变更清单/TDD记录/post-verify结果/遗留问题/可复用经验
code-review.md	CR	13节完整结构（见第六章）
test-report.md	TE	B类表格/C类表格/环境说明/结论
impact-analysis.md	SA(refactor)	变更波及域/受影响文件/风险评估
memory entry	归档时	tags/症状/根因/修复/防复发

11.3 模板的约束力

以 dev-log.md 为例，"TDD 记录"节的表格结构强制要求 Dev 填写：

```
| W-xx | 测试文件/用例 | 红（失败输出关键行） | 绿（通过输出关键行） |
```

这个四列表格的存在，使 Dev 无法用"已完成 TDD"这样的声明糊弄过去——必须填写具体的测试文件名、用例名、真实的失败输出和通过输出。CR 在审查时会核对这些内容的可信度。

十二、工程价值与效益分析

12.1 质量维度

需求追溯性：每行代码都可以追溯到 W-xx 工作项，W-xx 可以追溯到 R/S 编号，R/S 可以追溯到 proposal.md 的验收口径。这种完整的追溯链在传统开发中需要大量文档工作，Harness 通过模板和流程自动产生。

测试可信度：TDD 红绿证据 + CR 核查机制，使测试的可信度从”AI 声称写了测试”提升到”有证据证明测试曾经失败过”。

变更可审计性：dev-log.md 的变更清单精确到文件级别，每个变更都关联 R/W 编号。这使得”这行代码为什么在这里”有了可查的答案。

12.2 效率维度

上下文恢复效率：AI 代理通过读取 codebase-guide（约 5 分钟）即可恢复完整的代码库上下文，而不需要逐文件阅读（可能需要数小时）。

经验复用效率：memory/ 库中的历史经验可以在新任务开始时被检索，避免重复踩坑。例如”RTK 缓存在账号切换时需要 resetApiState”这条经验，一旦沉淀，后续所有相关任务都能受益。

审查效率：CR 的 13 节结构化报告使人工审查有了明确的检查点，而不是面对大量代码不知从何看起。

12.3 风险维度

技术债可见性：dev-log.md 的”已知遗留问题”节和 code-review.md 的”技术债”节，使技术债从隐性变为显性，并与具体任务关联，便于后续规划。

回归风险控制：C 类 E2E 测试（每次任务 1-2 条相关域回归）+ verify.sh 硬门禁，在不增加大量测试成本的情况下，对关键路径提供持续的回归保护。

知识流失风险：specs/ + memory/ 的持久化机制，使团队（包括 AI 代理）的知识不随人员（或对话）的更替而流失。

12.4 规模化维度

多任务并行：board.md 看板 + 任务目录隔离，支持多个任务同时进行，互不干扰。

多人协作：角色分离 + 契约文件，使多个人类或 AI 代理可以分别承担不同角色，协同完成一个任务。

跨项目移植：`.harness/multi-repo-adaptation.md` 提供了将 Harness 移植到新项目的详细指南，核心是复制 `.cursor/` + `.harness/` 两目录并按新技术栈调整约束。

十三、移植与扩展指南

13.1 移植到新项目

Harness 的设计与具体业务逻辑解耦，可以移植到任何技术栈的项目：

第一步：复制框架文件

```
cp -r .cursor/ /path/to/new-project/
cp -r .harness/ /path/to/new-project/
# 清空业务相关内容，保留框架结构
```

第二步：调整 code-standards.mdc 根据新项目的技术栈，修改代码规范约束（如将 ESM 改为 CommonJS，将 RTK Query 改为 React Query 等）。

第三步：调整 verify.sh 将 19 项检查中的技术栈特定检查（如 asyncHandler、RTK Query）替换为新项目的对应规范。

第四步：初始化 codebase-guide 运行 `codebase-guide-init.sh` 采集新项目的基础信息，然后由 SA 角色填写 backend-arch.md 和 frontend-arch.md。

第五步：建立初始 specs/ 将现有的业务需求按 BDD 格式整理到 specs/ 文件中，建立编号基线。

13.2 扩展新角色

如果项目需要额外的专业角色（如 DevOps 工程师、数据库管理员），可以按照以下步骤扩展：

1. 在 `.harness/agents/` 创建新角色契约文件
2. 在 `.cursor/skills/` 创建对应的 SKILL.md
3. 在 `harness-apply.md` 命令中添加新角色的调度逻辑
4. 在 `_actions_<role>.py` 中实现收工验证逻辑
5. 更新 `check-harness.sh` 以检查新角色文件的存在性

13.3 扩展新检查项

verify.sh 和 check-harness.sh 都设计为易于扩展：

```
# verify.sh 添加新检查项示例
# 20 禁止使用 any 类型（TypeScript 项目）
TS_ANY=$(grep -rn ': any' frontend/src --include='*.ts' --include='*.tsx'
[ -z "$TS_ANY" ]; check "前端无 any 类型" $?
```

十四、与同类方案的对比

14.1 与 OpenSpec 的对比

OpenSpec 是另一个面向 AI 代理的规格化方案，Harness 与其的主要差异：

维度	OpenSpec	Harness
交付验证	规格文档对齐	证据链（退出码+截图+行号）
操作标准化	提示词模板	命令+契约+SOP 三层
行为约束	规则声明	四层防线（软→硬）
测试体系	测试用例文档	TDD 红绿证据+E2E 取证
知识持久化	规格库	specs+guide+memory 三库
产出规范	自由格式	填表式模板（9 个）

14.2 与纯 Prompt Engineering 的对比

维度	纯 Prompt Engineering	Harness
约束强度	依赖 AI 遵守	进程外强制（hooks+scripts）
可绕过性	可以	不可以（退出码不可伪造）
知识持久化	无	三库持久化
可审计性	无	完整证据链
可移植性	低（与模型绑定）	高（与 IDE 事件系统绑定）
学习成本	低	中（需要理解框架）

14.3 与传统 CI/CD 的关系

Harness 不替代 CI/CD，而是与其互补：

- **CI/CD** 在代码提交后运行，是**事后验证**
- **Harness hooks** 在 AI 代理收工时运行，是**过程中验证**

Harness 的 verify.sh 可以直接集成到 CI/CD 流水线中，作为 PR 合并的门禁条件。

十五、结语与演进方向

15.1 核心价值重申

Harness 的本质是一套**工程化的信任机制**：在 AI 代理无法被完全信任的当下，通过流程设计、角色分离、进程外验证和证据链文化，构建一个**可信赖的 AI 辅助开发环境**。

它不追求 AI 的"完美"，而是接受 AI 的不完美，并通过系统设计将不完美的影响控制在可接受的范围内。

15.2 适用场景

Harness 特别适合以下场景：

- **功能交付要求高可靠性**的生产项目（金融、医疗、电商等）
- **团队规模较大**，需要多人（或多 AI 代理）协作的项目
- **技术债积累严重**，需要系统化管理的遗留项目

- **AI 代理使用频率高**，需要规范化管理 AI 行为的团队
- **合规要求严格**，需要完整审计链的项目

15.3 演进方向

基于当前框架，以下方向值得探索：

方向一：MCP 深度集成
将 Harness 的核心操作（init-task、stage-doc、verify、check-harness）封装为 MCP 工具，使 AI 代理可以直接调用，减少对 shell 命令的依赖。

方向二：多模型协同
不同角色使用不同的专业模型（如 BA 使用擅长需求分析的模型，CR 使用擅长代码审查的模型），通过 Harness 的角色契约统一接口。

方向三：自动化 Spec Merge
当前 Spec Merge 由 AI 代理执行，存在合并错误的风险。可以开发专门的 spec-merge 工具，基于编号规则自动合并，减少人工干预。

方向四：可视化看板
将 board.md 的文本看板升级为可视化 Web 界面，实时展示任务状态、证据链完整度、质量指标趋势。

方向五：跨 IDE 适配
当前 hooks 机制依赖 Cursor 的事件系统。可以开发适配层，使 Harness 的核心机制能在 VSCode、JetBrains 等其他 IDE 中运行。

附录 A：快速参考卡

A.1 三命令速查

```
/harness-propose <task-slug> <需求>    # 发起新任务
/harness-apply   <task-slug>           # 执行实施
/harness-archive <task-slug>           # 归档完成
```

A.2 验证命令速查

```
npm test                                    # 后端单测
npm run test:frontend                       # 前端单测
bash .harness/scripts/verify.sh             # 19 项硬门禁
bash .harness/scripts/check-harness.sh      # 133 项结构体检
bash .harness/scripts/backend-smoke.sh      # 后端冒烟
```

A.3 角色与产出速查

角色	产出	结论格式
BA	requirements.md	PASS / BLOCK
SA	design.md	PASS / BLOCK
RR	readiness-review.md	PASS / BLOCK
Dev	dev-log.md + 代码	退出码 0
CR	code-review.md	PASS / REJECT
TE	test-report.md + 截图	PASS / FAIL

A.4 Profile 速查

场景	Profile	propose 棒次
改文案/小修	quick	BA→SA
新功能	standard	BA→SA→RR
跨域重构	refactor	SA预扫→BA→SA→RR

附录 B：术语表

术语	定义
Harness	本框架的名称（缰绳/挽具），寓意对 AI 代理的约束与驾驭
证据链	不可伪造的外部证据集合（退出码+测试输出+截图+行号）
四层防线	rules → skills → hooks → scripts，约束强度递增
旁路验证	在 AI 进程之外执行的验证机制，AI 无法感知和干预
契约文件	agents/*.md，定义角色的职责、流程、禁止事项和收工条件
三库	specs（规格库）+ codebase-guide（代码知识库）+ memory（经验库）
Spec Merge	归档时将任务产生的新 Requirement/Scenario 并入 specs/ 库
Memory Merge	归档时将任务产生的可复用经验并入 memory/ 库
TDD 红绿证据	先写失败测试（红）再实现转绿，记录真实终端输出
profile	流程档位（quick/standard/refactor），决定 propose 棒次
人工卡点	两个需要人类明确批准才能继续的流程节点
W-xx	工作项编号，SA 在 design.md 中分配，Dev 按编号实现
R-xxx / S-xxx	Requirement/Scenario 编号，全库唯一，永不复用
退役列表	已废弃的 R/S 编号列表，占位防止复用
Testing-only	design.md 中专为 TE 写的视觉验收口径
心跳行	PM 输出的六类格式化状态行，禁止输出其他大段内容

本白皮书基于 Harness 框架 1.0 版本，根据三个培训视频（共约 79 分钟）的完整内容分析整理而成。